

COMONet: Formal Proofs of Shape Constraint Guarantees

Simon Frost

Contents

COMONet Basics	2
Positive weights via exp	2
Activation functions	2
Positive weights	2
ReLU activation function	2
Negative ReLU (concave unit activation)	3
Affine maps with positive weights	4
Monotonicity Theorems for COMONet	4
Key Results	4
Single neuron monotonicity	5
ReLU neuron monotonicity	5
Negative ReLU neuron monotonicity	6
Composition of monotone layers	6
Convexity Theorems for COMONet	7
Key Results	7
Convexity of ReLU	7
ReLU applied to affine map	7
Two-layer convexity	8
Concavity Theorems for COMONet	9
Key Results	9
Concavity of negative ReLU	9
ConcUnit	10
Multi-layer concavity	10
Composition Theorems for COMONet	12
Key Results	12
Composition rules	12
COMONet output layer	13
Full COMONet architecture theorems	13

```
import Mathlib.Analysis.SpecialFunctions.ExpDeriv
import Mathlib.Analysis.SpecialFunctions.Log.Basic
import Mathlib.Order.Monotone.Basic
import Mathlib.Topology.Order.Basic
```

COMONet Basics

Fundamental definitions and properties for the COMONet architecture.

Positive weights via exp

COMONet ensures positive weights by parameterizing weight matrices as $W = \exp(\tilde{W})$ where $\tilde{W} \in \mathbb{R}$ is unconstrained. Since $\exp : \mathbb{R} \rightarrow \mathbb{R}_+$, this guarantees $W > 0$ for all parameter values.

Activation functions

- **ReLU**: $\max(0, x)$ — non-decreasing, convex
- **Negative ReLU**: $-\max(0, -x) = \min(0, x)$ — non-decreasing, concave

```
namespace COMONetProofs
```

Positive weights

```
/-- The exponential function is strictly positive for all real inputs.
This is the foundational property ensuring COMONet weights are positive. -/
theorem exp_pos (w : ℝ) : Real.exp w > 0 :=
  Real.exp_pos w

/-- The exponential function is positive (non-strict), useful for
weight multiplication properties. -/
theorem exp_nonneg (w : ℝ) : Real.exp w ≥ 0 :=
  le_of_lt (Real.exp_pos w)
```

ReLU activation function

```
/-- ReLU activation: `relu(x) = max(0, x)` -/
noncomputable def relu (x : ℝ) : ℝ := max 0 x

/-- ReLU is non-negative. -/
```

```

theorem relu_nonneg (x : ℝ) : relu x ≥ 0 := by
  simp [relu]

/-- ReLU at non-negative input equals the input. -/
theorem relu_of_nonneg {x : ℝ} (hx : x ≥ 0) : relu x = x := by
  simp [relu, max_eq_right (by linarith : (0 : ℝ) ≤ x)]

/-- ReLU at negative input is zero. -/
theorem relu_of_neg {x : ℝ} (hx : x < 0) : relu x = 0 := by
  simp [relu, max_eq_left (by linarith : (0 : ℝ) ≥ x)]

/-- ReLU is monotone non-decreasing. -/
theorem relu_monotone : Monotone relu := by
  intro a b hab
  simp only [relu]
  exact max_le_max_left 0 hab

```

Negative ReLU (concave unit activation)

```

/-- Negative ReLU: `nrelu(x) = -max(0, -x) = min(0, x)`.
Used in COMONet's concave units. -/
noncomputable def nrelu (x : ℝ) : ℝ := -relu (-x)

/-- Negative ReLU equals min(0, x). -/
theorem nrelu_eq_min (x : ℝ) : nrelu x = min 0 x := by
  simp only [nrelu, relu]
  -- -max(0, -x) = min(0, x) by the identity -max(a,b) = min(-a,-b)
  cases le_or_gt 0 x with
  | inl h =>
    -- x ≥ 0: max(0,-x) = 0, min(0,x) = 0
    have : -x ≤ 0 := neg_nonpos.mpr h
    rw [max_eq_left this, neg_zero, min_eq_left h]
  | inr h =>
    -- x < 0: max(0,-x) = -x, min(0,x) = x
    have : 0 ≤ -x := neg_nonneg.mpr (le_of_lt h)
    rw [max_eq_right this, neg_neg, min_eq_right (le_of_lt h)]

/-- Negative ReLU is non-positive. -/
theorem nrelu_nonpos (x : ℝ) : nrelu x ≤ 0 := by
  rw [nrelu_eq_min]
  exact min_le_left 0 x

/-- Negative ReLU is monotone non-decreasing. -/
theorem nrelu_monotone : Monotone nrelu := by

```

```

intro a b hab
simp only [nrelu]
exact neg_le_neg (relu_monotone (neg_le_neg hab))

```

Affine maps with positive weights

```

/-- An affine map  $x \mapsto w * x + b$  with  $w > 0$  is strictly monotone. -/
theorem affine_pos_weight_strictMono {w b : ℝ} (hw : w > 0) :
  StrictMono (fun x => w * x + b) := by
  intro a c hac
  linarith [mul_lt_mul_of_pos_left hac hw]

/-- An affine map  $x \mapsto w * x + b$  with  $w > 0$  is monotone non-decreasing. -/
theorem affine_pos_weight_mono {w b : ℝ} (hw : w > 0) :
  Monotone (fun x => w * x + b) :=
  StrictMono.monotone (affine_pos_weight_strictMono hw)

/-- An affine map with  $w \geq 0$  is monotone non-decreasing. -/
theorem affine_nonneg_weight_mono {w b : ℝ} (hw : w ≥ 0) :
  Monotone (fun x => w * x + b) := by
  intro a c hac
  have : w * a ≤ w * c := mul_le_mul_of_nonneg_left hac hw
  linarith

end COMONetProofs

import COMONetProofs.Basics
import Mathlib.Order.Monotone.Basic

```

Monotonicity Theorems for COMONet

Key Results

1. **Single neuron monotonicity:** If $w > 0$ and σ is monotone non-decreasing, then $x \mapsto \sigma(w * x + b)$ is monotone non-decreasing.
2. **exp-weight monotonicity:** For any unconstrained $\tilde{w} \in \mathbb{R}$, $x \mapsto \sigma(\exp(\tilde{w}) * x + b)$ is monotone when σ is monotone.
3. **Composition preservation:** The composition of monotone non-decreasing functions is monotone non-decreasing.

These theorems form the basis of COMONet's monotonicity guarantee: each layer uses $\exp(\tilde{w})$ weights (positive by construction) and monotone activations, so the overall network is monotone.

```
namespace COMONetProofs
```

Single neuron monotonicity

```
-- **Core monotonicity theorem**: If  $\sigma$  is monotone non-decreasing and  $w > 0$ , then the neuron  $x \mapsto \sigma(w * x + b)$  is monotone non-decreasing.
```

This is the fundamental building block of COMONet's monotonicity guarantee. The proof follows from:

1. $x \mapsto w * x + b$ is monotone when $w > 0$

2. Monotone $\circ$ Monotone = Monotone -/

```
theorem neuron_monotone_of_pos_weight
```

```
{ $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ } (h $\sigma$  : Monotone  $\sigma$ ) {w b :  $\mathbb{R}$ } (hw : w > 0) :
```

```
Monotone (fun x =>  $\sigma$  (w * x + b)) :=
```

```
h $\sigma$ .comp (affine_pos_weight_mono hw)
```

```
-- **exp-weight monotonicity**: For any unconstrained weight  $\tilde{w} \in \mathbb{R}$  and any monotone activation  $\sigma$ , the neuron  $x \mapsto \sigma(\exp(\tilde{w}) * x + b)$  is monotone non-decreasing.
```

This is the key insight of COMONet: parameterizing weights through $\exp$ guarantees positivity without constrained optimization, and combined with monotone activations gives guaranteed monotone neurons. -/

```
theorem neuron_monotone_of_exp_weight
```

```
{ $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ } (h $\sigma$  : Monotone  $\sigma$ ) (w_tilde b :  $\mathbb{R}$ ) :
```

```
Monotone (fun x =>  $\sigma$  (Real.exp w_tilde * x + b)) :=
```

```
neuron_monotone_of_pos_weight h $\sigma$  (exp_pos w_tilde)
```

ReLU neuron monotonicity

```
-- A ReLU neuron with positive weight is monotone. -/
```

```
theorem relu_neuron_monotone {w b :  $\mathbb{R}$ } (hw : w > 0) :
```

```
Monotone (fun x => relu (w * x + b)) :=
```

```
neuron_monotone_of_pos_weight relu_monotone hw
```

```
-- A ReLU neuron with exp-weight is monotone for any unconstrained weight. -/
```

```
theorem relu_neuron_exp_weight_monotone (w_tilde b :  $\mathbb{R}$ ) :
```

```
Monotone (fun x => relu (Real.exp w_tilde * x + b)) :=
```

```
neuron_monotone_of_exp_weight relu_monotone w_tilde b
```

Negative ReLU neuron monotonicity

```
-- A negative ReLU neuron with positive weight is monotone. -/
theorem nrelu_neuron_monotone {w b : ℝ} (hw : w > 0) :
  Monotone (fun x => nrelu (w * x + b)) :=
  neuron_monotone_of_pos_weight nrelu_monotone hw

-- A negative ReLU neuron with exp-weight is monotone. -/
theorem nrelu_neuron_exp_weight_monotone (w_tilde b : ℝ) :
  Monotone (fun x => nrelu (Real.exp w_tilde * x + b)) :=
  neuron_monotone_of_exp_weight nrelu_monotone w_tilde b
```

Composition of monotone layers

```
-- **Monotone composition**: The composition of two monotone non-decreasing
functions is monotone non-decreasing.
```

This is a standard result, but we state it explicitly for clarity in the COMONet context: if layer L_1 and layer L_2 are both monotone, then $L_2 \circ L_1$ is monotone. -/

```
theorem monotone_comp_monotone
  {f g : ℝ → ℝ} (hf : Monotone f) (hg : Monotone g) :
  Monotone (g ∘ f) :=
  hg.comp hf
```

```
-- **Multi-layer monotonicity (3 layers)**: Composition of three
monotone functions is monotone.
```

COMONet typically uses 2-3 hidden layers; this covers the 3-layer case (input $\rightarrow$ hidden₁ $\rightarrow$ hidden₂ $\rightarrow$ output). -/

```
theorem monotone_comp3
  {f g h : ℝ → ℝ} (hf : Monotone f) (hg : Monotone g) (hh : Monotone h) :
  Monotone (h ∘ g ∘ f) :=
  hh.comp (hg.comp hf)
```

```
-- **COMONet ExpUnit (single layer)**: A single COMONet monotone layer
with exp-parameterized weight, bias, and monotone activation. -/
```

```
theorem comonet_exp_unit_monotone
  {σ : ℝ → ℝ} (hσ : Monotone σ) (w_tilde b : ℝ) :
  Monotone (fun x => σ (Real.exp w_tilde * x + b)) :=
  neuron_monotone_of_exp_weight hσ w_tilde b
```

```
-- **COMONet two-layer monotone network**: Two sequential exp-weight
layers with monotone activations produce a monotone function. -/
theorem comonet_two_layer_monotone
```

```

    { $\sigma_1$   $\sigma_2$  :  $\mathbb{R} \rightarrow \mathbb{R}$ } (h $\sigma_1$  : Monotone  $\sigma_1$ ) (h $\sigma_2$  : Monotone  $\sigma_2$ )
    (w1 b1 w2 b2 :  $\mathbb{R}$ ) :
    Monotone (fun x =>
       $\sigma_2$  (Real.exp w2 *  $\sigma_1$  (Real.exp w1 * x + b1) + b2)) :=
    (neuron_monotone_of_exp_weight h $\sigma_2$  w2 b2).comp
    (neuron_monotone_of_exp_weight h $\sigma_1$  w1 b1)

end COMONetProofs

```

```

import COMONetProofs.Basics
import Mathlib.Analysis.Convex.Function
import Mathlib.Analysis.Convex.SpecificFunctions.Basic

```

Convexity Theorems for COMONet

Key Results

1. **ReLU is convex:** $\max(0, x)$ is a convex function on $\mathbb{R}$.
2. **COMONet ConvUnit:** $\text{relu}(\exp(\tilde{w}) * x + b)$ is convex in x .
3. **Multi-layer convexity:** Composition of ConvUnit layers is convex.

```

namespace COMONetProofs

```

Convexity of ReLU

```

/-- ReLU is convex on  $\mathbb{R}$ . Follows from `max` of two convex functions. -/
theorem relu_convex0n : Convex0n  $\mathbb{R}$  Set.univ relu := by
  apply Convex0n.sup
  · exact convex0n_const 0 convex_univ
  · exact convex0n_id convex_univ

```

ReLU applied to affine map

```

/-- Helper: affine precomposition preserves convexity when
the original function is convex. We prove `relu ∘ affine` is convex
directly from the definition. -/
theorem relu_affine_convex0n (w b :  $\mathbb{R}$ ) :
  Convex0n  $\mathbb{R}$  Set.univ (fun x => relu (w * x + b)) := by
  rw [Convex0n]
  refine {convex_univ, ?_}
  intro x _ y _ a c ha hc hac
  simp only [relu, smul_eq_mul]

```

```

-- Goal:  $\max 0 (w * (a * x + c * y) + b) \leq a * \max 0 (w * x + b) + c * \max 0 (w * y + b)$ 
have h_lin :  $w * (a * x + c * y) + b = a * (w * x + b) + c * (w * y + b)$  := by
  have hac1 :  $a + c = 1$  := hac
  have :  $b = (a + c) * b$  := by rw [hac1, one_mul]
  linarith
rw [h_lin]
exact max_le
  (add_nonneg (mul_nonneg ha (le_max_left 0 _)) (mul_nonneg hc (le_max_left 0 _)))
  (add_le_add (mul_le_mul_of_nonneg_left (le_max_right 0 _) ha)
    (mul_le_mul_of_nonneg_left (le_max_right 0 _) hc))

/-- **COMONet ConvUnit with exp-weight**: For any unconstrained weight
`w` and bias `b`, `relu(exp(w) * x + b)` is convex in `x`. -/
theorem relu_exp_weight_convex0n (w_tilde b : ℝ) :
  Convex0n ℝ Set.univ (fun x => relu (Real.exp w_tilde * x + b)) :=
  relu_affine_convex0n (Real.exp w_tilde) b

```

Two-layer convexity

```

/-- **Two-layer ConvUnit convexity**: `relu(w2 * relu(w1 * x + b1) + b2)`
is convex when `w1, w2 > 0`. -/
theorem two_layer_convUnit_convex0n (w1 b1 w2 b2 : ℝ)
  (hw2 : w2 ≥ 0) :
  Convex0n ℝ Set.univ
    (fun x => relu (w2 * relu (w1 * x + b1) + b2)) := by
  rw [Convex0n]
  refine {convex_univ, ?_}
  intro x _ y _ a c ha hc hac
  simp only [relu, smul_eq_mul]
  -- Inner convexity
  have h_inner := (relu_affine_convex0n w1 b1).2
  (Set.mem_univ x) (Set.mem_univ y) ha hc hac
  simp only [relu, smul_eq_mul] at h_inner
  set ix := max 0 (w1 * x + b1)
  set iy := max 0 (w1 * y + b1)
  set iz := max 0 (w1 * (a * x + c * y) + b1)
  -- h_inner : iz ≤ a * ix + c * iy
  -- Need:  $\max 0 (w_2 * iz + b_2) \leq a * \max 0 (w_2 * ix + b_2) + c * \max 0 (w_2 * iy + b_2)$ 
  have h_w2_iz :  $w_2 * iz \leq w_2 * (a * ix + c * iy)$  :=
    mul_le_mul_of_nonneg_left h_inner hw2
  have h_expand :  $w_2 * (a * ix + c * iy) = a * (w_2 * ix) + c * (w_2 * iy)$  := by ring
  have h_sum :  $w_2 * iz + b_2 \leq a * (w_2 * ix + b_2) + c * (w_2 * iy + b_2)$  := by
    have hac1 :  $a + c = 1$  := hac
    have h1 :  $w_2 * iz \leq a * (w_2 * ix) + c * (w_2 * iy)$  := by linarith [h_w2_iz, h_expand]

```



```

    have h2 : a * (w2 * ix + b2) + c * (w2 * iy + b2)
      = a * (w2 * ix) + c * (w2 * iy) + (a + c) * b2 := by ring
    rw [h2, hac1, one_mul]
    linarith
  exact max_le
    (add_nonneg (mul_nonneg ha (le_max_left 0 _)) (mul_nonneg hc (le_max_left 0 _)))
    (le_trans h_sum
      (add_le_add (mul_le_mul_of_nonneg_left (le_max_right 0 _) ha)
        (mul_le_mul_of_nonneg_left (le_max_right 0 _) hc)))

/-- **Two-layer ConvUnit with exp-weights** is convex. -/
theorem two_layer_convUnit_exp_weight_convexOn (w1 b1 w2 b2 : ℝ) :
  ConvexOn ℝ Set.univ
    (fun x => relu (Real.exp w2 * relu (Real.exp w1 * x + b1) + b2)) :=
  two_layer_convUnit_convexOn (Real.exp w1) b1 (Real.exp w2) b2
    (le_of_lt (exp_pos w2))

end COMONetProofs

import COMONetProofs.Basics
import COMONetProofs.Convexity
import Mathlib.Analysis.Convex.Function

```

Concavity Theorems for COMONet

Key Results

1. **Negative ReLU is concave:** $\text{nrelu}(x) = -\text{relu}(-x) = \min(0, x)$ is concave.
2. **COMONet ConcUnit:** $\text{nrelu}(\exp(\tilde{w}) * x + b)$ is concave in x .
3. **Multi-layer concavity:** ConcUnit layers with positive weights preserve concavity.

```
namespace COMONetProofs
```

Concavity of negative ReLU

```

/-- Negative ReLU is concave on ℝ. -/
theorem nrelu_concaveOn : ConcaveOn ℝ Set.univ nrelu := by
  rw [ConcaveOn]
  refine {convex_univ, ?_}
  intro x _ y _ a c ha hc hac
  simp only [nrelu, relu, smul_eq_mul]
  -- Need: a * (-max 0 (-x)) + c * (-max 0 (-y)) ≤ -(max 0 (-(a*x + c*y)))

```

```

-- i.e.,  $\max 0 (-(a*x+c*y)) \leq a * \max 0 (-x) + c * \max 0 (-y)$ 
-- This is convexity of relu applied to negated inputs
suffices h :  $\max 0 (-(a * x + c * y)) \leq a * \max 0 (-x) + c * \max 0 (-y)$  by linarith
have h_neg :  $-(a * x + c * y) = a * (-x) + c * (-y)$  := by ring
rw [h_neg]
exact max_le
  (add_nonneg (mul_nonneg ha (le_max_left 0 _)) (mul_nonneg hc (le_max_left 0 _)))
  (add_le_add (mul_le_mul_of_nonneg_left (le_max_right 0 _) ha)
    (mul_le_mul_of_nonneg_left (le_max_right 0 _) hc))

```

ConcUnit

```

/-- **COMONet ConcUnit**: `nrelu(w * x + b)` is concave for any `w, b`. -/
theorem nrelu_affine_concaveOn (w b : ℝ) :
  ConcaveOn ℝ Set.univ (fun x => nrelu (w * x + b)) := by
  rw [ConcaveOn]
  refine {convex_univ, ?_}
  intro x _ y _ a c ha hc hac
  simp only [nrelu, relu, smul_eq_mul]
  -- Need:  $a * (-\max 0 (-(w*x+b))) + c * (-\max 0 (-(w*y+b)))$ 
  --  $\leq -(\max 0 (-(w*(a*x+c*y)+b)))$ 
  suffices h :  $\max 0 (-(w * (a * x + c * y) + b))$ 
     $\leq a * \max 0 (-(w * x + b)) + c * \max 0 (-(w * y + b))$  by linarith
  have h_lin :  $-(w * (a * x + c * y) + b) = a * (-(w * x + b)) + c * (-(w * y + b))$  := by
    have hac1 :  $a + c = 1$  := hac
    have :  $b = (a + c) * b$  := by rw [hac1, one_mul]
    linarith
  rw [h_lin]
  exact max_le
    (add_nonneg (mul_nonneg ha (le_max_left 0 _)) (mul_nonneg hc (le_max_left 0 _)))
    (add_le_add (mul_le_mul_of_nonneg_left (le_max_right 0 _) ha)
      (mul_le_mul_of_nonneg_left (le_max_right 0 _) hc))

/-- **COMONet ConcUnit with exp-weight**: For any unconstrained weight
`w` and bias `b`, `nrelu(exp(w) * x + b)` is concave in `x`. -/
theorem nrelu_exp_weight_concaveOn (w_tilde b : ℝ) :
  ConcaveOn ℝ Set.univ (fun x => nrelu (Real.exp w_tilde * x + b)) :=
  nrelu_affine_concaveOn (Real.exp w_tilde) b

```

Multi-layer concavity

```

/-- **Two-layer ConcUnit concavity** with positive outer weight. -/
theorem two_layer_concUnit_concaveOn (w1 b1 w2 b2 : ℝ)
  (hw2 : w2 ≥ 0) :

```

```

    ConcaveOn ℝ Set.univ
      (fun x => nrelu (w₂ * nrelu (w₁ * x + b₁) + b₂)) := by
rw [ConcaveOn]
refine (convex_univ, ?_)
intro x _ y _ a c ha hc hac
simp only [nrelu, relu, smul_eq_mul]
-- Inner concavity: nrelu(w₁*z+b₁) is concave
have h_inner := (nrelu_affine_concaveOn w₁ b₁).2
  (Set.mem_univ x) (Set.mem_univ y) ha hc hac
simp only [nrelu, relu, smul_eq_mul] at h_inner
set ix := -max 0 (-(w₁ * x + b₁))
set iy := -max 0 (-(w₁ * y + b₁))
set iz := -max 0 (-(w₁ * (a * x + c * y) + b₁))
-- h_inner : a * ix + c * iy ≤ iz
-- Need outer nrelu to preserve: suffices to show
-- max 0 (-(w₂*iz+b₂)) ≤ a * max 0 (-(w₂*ix+b₂)) + c * max 0 (-(w₂*iy+b₂))
suffices h : max 0 (-(w₂ * iz + b₂))
  ≤ a * max 0 (-(w₂ * ix + b₂)) + c * max 0 (-(w₂ * iy + b₂)) by linarith
-- Since iz ≥ a*ix + c*iy and w₂ ≥ 0:
-- w₂*iz ≥ w₂*(a*ix + c*iy) = a*w₂*ix + c*w₂*iy
-- So -(w₂*iz+b₂) ≤ -(a*w₂*ix + c*w₂*iy + b₂) = a*(-(w₂*ix+b₂)) + c*(-(w₂*iy+b₂))
have h_w2_iz : w₂ * (a * ix + c * iy) ≤ w₂ * iz :=
  mul_le_mul_of_nonneg_left h_inner hw₂
have h_neg : -(w₂ * iz + b₂) ≤ a * (-(w₂ * ix + b₂)) + c * (-(w₂ * iy + b₂)) := by
  have hac1 : a + c = 1 := hac
  have : b₂ = (a + c) * b₂ := by rw [hac1, one_mul]
  linarith
calc max 0 (-(w₂ * iz + b₂))
  ≤ max 0 (a * (-(w₂ * ix + b₂)) + c * (-(w₂ * iy + b₂))) :=
  max_le_max_left 0 h_neg
_ ≤ a * max 0 (-(w₂ * ix + b₂)) + c * max 0 (-(w₂ * iy + b₂)) :=
  max_le
  (add_nonneg (mul_nonneg ha (le_max_left 0 _)) (mul_nonneg hc (le_max_left 0 _)))
  (add_le_add (mul_le_mul_of_nonneg_left (le_max_right 0 _) ha)
    (mul_le_mul_of_nonneg_left (le_max_right 0 _) hc))

/-- **Two-layer ConcUnit with exp-weights** is concave. -/
theorem two_layer_concUnit_exp_weight_concaveOn (w₁ b₁ w₂ b₂ : ℝ) :
  ConcaveOn ℝ Set.univ
    (fun x => nrelu (Real.exp w₂ * nrelu (Real.exp w₁ * x + b₁) + b₂)) :=
  two_layer_concUnit_concaveOn (Real.exp w₁) b₁ (Real.exp w₂) b₂
  (le_of_lt (exp_pos w₂))
end COMONetProofs

```

```

import COMONetProofs.Basics
import COMONetProofs.Monotonicity
import COMONetProofs.Convexity
import COMONetProofs.Concavity
import Mathlib.Analysis.Convex.Function
import Mathlib.Order.Monotone.Basic

```

Composition Theorems for COMONet

Key Results

1. **Monotone $\circ$ Monotone = Monotone**: Standard composition rule.
2. **Positive scaling preserves convexity/concavity**.
3. **COMONet full network guarantees**: 2-hidden + output layer.
4. **Summary theorem**: Collects all architectural guarantees.

```
namespace COMONetProofs
```

Composition rules

```

/-- **Monotone composition** : standard mathematical fact. -/
theorem comp_monotone {f g : ℝ → ℝ}
  (hf : Monotone f) (hg : Monotone g) :
  Monotone (g ∘ f) :=
  hg.comp hf

/-- Positive scaling preserves convexity. -/
theorem pos_scale_convex0n {f : ℝ → ℝ}
  (hf : Convex0n ℝ Set.univ f) {α : ℝ} (hα : α ≥ 0) (b : ℝ) :
  Convex0n ℝ Set.univ (fun x => α * f x + b) := by
  rw [Convex0n]
  refine {convex_univ, ?_}
  intro x _ y _ a c ha hc hac
  simp only [smul_eq_mul]
  have hf_ineq := hf.2 (Set.mem_univ x) (Set.mem_univ y) ha hc hac
  simp only [smul_eq_mul] at hf_ineq
  have h1 : α * f (a * x + c * y) ≤ α * (a * f x + c * f y) :=
    mul_le_mul_of_nonneg_left hf_ineq hα
  have hac1 : a + c = 1 := hac
  have : b = (a + c) * b := by rw [hac1, one_mul]
  linarith

/-- Positive scaling preserves concavity. -/

```

```

theorem pos_scale_concaveOn {f : ℝ → ℝ}
  (hf : ConcaveOn ℝ Set.univ f) {α : ℝ} (hα : α ≥ 0) (b : ℝ) :
  ConcaveOn ℝ Set.univ (fun x => α * f x + b) := by
  rw [ConcaveOn]
  refine {convex_univ, ?_}
  intro x _ y _ a c ha hc hac
  simp only [smul_eq_mul]
  have hf_ineq := hf.2 (Set.mem_univ x) (Set.mem_univ y) ha hc hac
  simp only [smul_eq_mul] at hf_ineq
  have h1 : α * (a * f x + c * f y) ≤ α * f (a * x + c * y) :=
    mul_le_mul_of_nonneg_left hf_ineq hα
  have hac1 : a + c = 1 := hac
  have : b = (a + c) * b := by rw [hac1, one_mul]
  linarith

```

COMONet output layer

```

/-- **COMONet linear output with exp-weight preserves convexity**. -/
theorem exp_weight_output_preserves_convexOn {f : ℝ → ℝ}
  (hf : ConvexOn ℝ Set.univ f) (w_tilde b : ℝ) :
  ConvexOn ℝ Set.univ (fun x => Real.exp w_tilde * f x + b) :=
  pos_scale_convexOn hf (le_of_lt (exp_pos w_tilde)) b

/-- **COMONet linear output with exp-weight preserves concavity**. -/
theorem exp_weight_output_preserves_concaveOn {f : ℝ → ℝ}
  (hf : ConcaveOn ℝ Set.univ f) (w_tilde b : ℝ) :
  ConcaveOn ℝ Set.univ (fun x => Real.exp w_tilde * f x + b) :=
  pos_scale_concaveOn hf (le_of_lt (exp_pos w_tilde)) b

```

Full COMONet architecture theorems

```

/-- **COMONet monotone network (2 hidden + output)**: Monotone for any weights. -/
theorem comonet_monotone_network_3layer
  (w1 b1 w2 b2 w3 b3 : ℝ) :
  Monotone (fun x =>
    Real.exp w3 * relu (Real.exp w2 * relu (Real.exp w1 * x + b1) + b2) + b3) :=
  (affine_nonneg_weight_mono (le_of_lt (exp_pos w3))).comp
  (relu_monotone.comp
    ((affine_nonneg_weight_mono (le_of_lt (exp_pos w2))).comp
      (relu_monotone.comp (affine_nonneg_weight_mono (le_of_lt (exp_pos w1))))))

/-- **COMONet convex network (2 hidden + output)**: Convex for any weights. -/
theorem comonet_convex_network_3layer
  (w1 b1 w2 b2 w3 b3 : ℝ) :

```

```

ConvexOn ℝ Set.univ (fun x =>
  Real.exp w3 * relu (Real.exp w2 * relu (Real.exp w1 * x + b1) + b2) + b3) :=
exp_weight_output_preserves_convexOn
  (two_layer_convUnit_exp_weight_convexOn w1 b1 w2 b2)
  w3 b3

/-- **COMONet concave network (2 hidden + output)**: Concave for any weights. -/
theorem comonet_concave_network_3layer
  (w1 b1 w2 b2 w3 b3 : ℝ) :
  ConcaveOn ℝ Set.univ (fun x =>
    Real.exp w3 * nrelu (Real.exp w2 * nrelu (Real.exp w1 * x + b1) + b2) + b3) :=
exp_weight_output_preserves_concaveOn
  (two_layer_concUnit_exp_weight_concaveOn w1 b1 w2 b2)
  w3 b3

/-- **Summary theorem**: COMONet architectural constraints guarantee
shape properties for ALL parameter values. No constrained optimization
needed – the guarantees are structural. -/
theorem comonet_shape_guarantees :
  -- 1. Monotonicity: for all weights, the monotone network is monotone
  (∀ w1 b1 w2 b2 : ℝ,
    Monotone (fun x => relu (Real.exp w2 * relu (Real.exp w1 * x + b1) + b2))) ∧
  -- 2. Convexity: for all weights, the convex network is convex
  (∀ w1 b1 w2 b2 : ℝ,
    ConvexOn ℝ Set.univ
      (fun x => relu (Real.exp w2 * relu (Real.exp w1 * x + b1) + b2))) ∧
  -- 3. Concavity: for all weights, the concave network is concave
  (∀ w1 b1 w2 b2 : ℝ,
    ConcaveOn ℝ Set.univ
      (fun x => nrelu (Real.exp w2 * nrelu (Real.exp w1 * x + b1) + b2))) := by
exact {
  fun w1 b1 w2 b2 =>
    relu_monotone.comp
      ((affine_nonneg_weight_mono (le_of_lt (exp_pos w2))).comp
        (relu_monotone.comp (affine_nonneg_weight_mono (le_of_lt (exp_pos w1))))),
  fun w1 b1 w2 b2 => two_layer_convUnit_exp_weight_convexOn w1 b1 w2 b2,
  fun w1 b1 w2 b2 => two_layer_concUnit_exp_weight_concaveOn w1 b1 w2 b2
}
end COMONetProofs

```